

VMI-FGSM Attack in Face Recognition

Khushi

June 27, 2026

Attacks Considered

- VMI-FGSM — Variance-tuned Momentum Iterative FGSM (CVPR 2021)
- MI-FGSM (momentum-based baseline)
- SI-NI-FGSM (strongest existing baseline)

Selecting VMI-FGSM

Why VMI-FGSM?

- Recent CVPR 2021 publication
- Official implementation available in TransferAttack repository
- Not implemented in intern baseline
- Designed specifically to improve transferability of adversarial examples
- Targets CNN-based models — matches our face recognition setup

Code

```
github.com/Khushi250321/transferattackInterns
```

Understanding VMI-FGSM

VMI-FGSM = Variance-tuned Momentum Iterative FGSM

Gradient Variance Estimation

- Samples N random neighbors in $\beta \cdot \epsilon$ neighborhood
- Computes gradient at each neighbor
- Estimates variance: $v = (\sum g_i - \text{grad}) / N$
- Variance captures gradient instability

Momentum Update

- Like MI-FGSM, maintains accumulated gradient
- $\text{grad_tuned} = \text{grad} + v$
- $g = \text{decay} * g + \text{grad_tuned}$
- $\text{adv} = \text{adv} + \alpha * \text{sign}(g)$

Attack Objectives (unchanged)

- Impersonation: maximize cosine similarity
- Dodging: minimize cosine similarity

Difference From MI-FGSM

- Adds gradient variance estimation step
- Tunes update direction using variance
- More stable optimization path
- Specifically designed to improve transferability

Attack Implementation

Pseudocode :

```
def vmi_fgsm(model, x, tgt_emb,
             attack_type, beta=1.5, n=20):

    adv = tf.identity(x)
    g = tf.zeros_like(x)
    v = tf.zeros_like(x)
    alpha = EPSILON / NUM_ITER

    for _ in range(NUM_ITER):
        grad = compute_grad(adv)
        grad_var = zeros_like(grad)
        for _ in range(n):
            nb = adv + uniform(
                -beta*eps, beta*eps)
            g2 = compute_grad(nb)
            grad_var += g2 - grad
        v = grad_var / n
        grad_t = grad + v
        grad_t /= mean(abs(grad_t))
        g = DECAY * g + grad_t
        adv += alpha * sign(g)
        adv = clip(adv, x-eps, x+eps)
        adv = clip(adv, -1.0, 1.0)
    return adv
```

VMI-FGSM in Our Implementation

1. Initialization without random start
2. Step size $\alpha = \epsilon / \text{iterations}$ (NUM_ITER=10)
3. N=20 neighbor samples per iteration
4. Beta=1.5 controls neighborhood size
5. Variance v used to tune gradient direction
6. Momentum accumulation (DECAY=1.0)
7. Explicit ϵ -ball projection via clipping

Code Integration

Step 1 — Register VMI_FGSM

```
ALL_ATTACKS = [  
    PGD, MI_FGSM,  
    TI_FGSM, SI_NI_FGSM,  
    MI_ADMIX_DI_TI,  
    ATT, PGN,  
    VMI_FGSM # <-- added  
]
```

Step 3 — Dispatcher

```
if attack_name == "VMI_FGSM":  
    return vmi_fgsm(...)
```

Step 2 — vmi_fgsm() Function

1. Initialize adversarial image
2. Compute embedding similarity loss
3. Compute gradient at current point
4. Sample N neighbors, compute variance
5. Tune gradient with variance
6. Update momentum g
7. Project into ϵ -ball
8. Clip image to valid range
9. Repeat for NUM_ITER = 10

Key Challenges Encountered

Threshold Bug

- Earlier ATT script used hardcoded DigiFace threshold (0.5)
- Required dataset-specific thresholds (lfw, celeba, vggface2)
- Fixed by using row['dataset'] to look up correct threshold

Official Pipeline Integration

- build_subset_baselines.py did not include VMI_FGSM in ATTACKS list
- Added VMI_FGSM to ATTACKS list in script
- Fixed row_id mismatch between subset CSV and pipeline

CSV Format Mismatch

- Custom script output different columns than official pipeline
- Wrote convert_vmi_to_official.py to reformat results
- Combined with baseline CSV before running official pipeline

Slow Execution

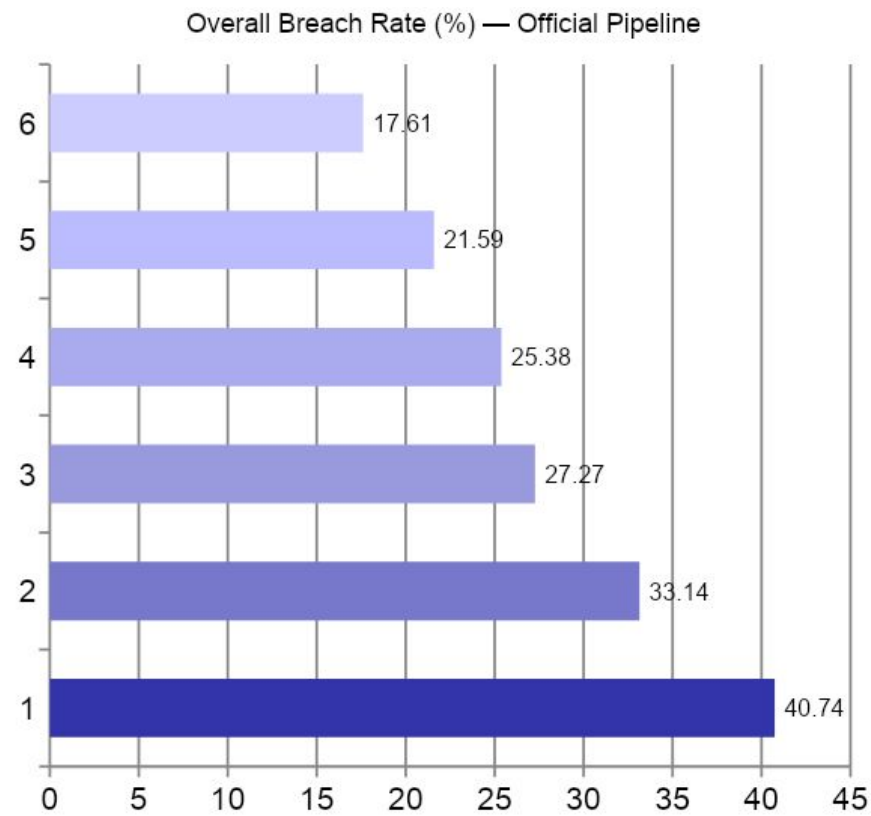
- VMI-FGSM samples N=20 neighbors per iteration
- Much slower than MI-FGSM (20x more gradient calls)
- Acceptable trade-off for improved transferability

Final Results

Attack	Breach Rate
VMI-FGSM (ours)	40.74% ✓
SI_NI_FGSM	33.14%
MI_FGSM	27.27%
MI_ADMIX_DI_TI	25.38%
TI_FGSM	21.59%
PGD	17.61%

By Attack Type:

Type	Breach Rate
Dodging	51.96%
Impersonation	24.51%



What I Learned

Technical

- Transfer attack mechanics and evaluation pipeline
- How gradient variance improves adversarial transferability
- Adapting NeurIPS papers to a new problem setting
- Importance of using official evaluation pipeline for fair comparison
- Debugging threshold and row_id mismatches in evaluation code

Process

- Integrating new attacks into existing codebases
- Converting custom results to match official CSV schema
- Verifying results through official pipeline before reporting
- Git workflow — syncing fork with upstream repo
- Iterating through multiple attacks (ATT → PGN → VMI-FGSM) to find best result

Thank You

Thank You